TurtleBot3 Autonomous Navigation Pipeline

Overview

This project implements an autonomous navigation pipeline for a TurtleBot3 mobile robot in a simulated indoor environment. The system connects ROS 2 communication, occupancy-map based planning, path optimization, and trajectory tracking into a single workflow that can be launched from MATLAB while using Gazebo and RViz for simulation and visualization.

The goal of the project is to drive a differential-drive robot from its current odometry pose to an operator-selected RViz goal while avoiding occupied map regions and producing velocity commands on /cmd_vel.

System Architecture

The implementation is organized around four layers:

* ROS 2 interface: subscribes to odometry and RViz goal poses, and publishes velocity commands for the robot.
* Map processing: loads the indoor occupancy map, aligns it with the simulator coordinate frame, and inflates obstacles for planning clearance.
* Global planning: generates a collision-free path using sampling-based planners such as PRM, RRT, and BiRRT.
* Local tracking: optimizes the planned path and follows it with a Time Elastic Band controller.

The ROS 2 assets under ros2_ws/ provide the corresponding Gazebo world, robot spawn launch file, Nav2 localization launch file, and map configuration.

Planning Method

The final configuration uses a bidirectional RRT planner as the primary global planner. BiRRT is suitable for this type of indoor map because it grows search trees from both the start and goal states, which can reduce search time in cluttered environments compared with a single-tree RRT.

The planner operates in SE(2), so each state contains x, y, and heading. Collision checking is performed against a binary occupancy map. The path is then post-processed with MATLAB’s path optimization tools to reduce unnecessary detours before trajectory tracking.

Control Method

The optimized global path is passed to a Time Elastic Band controller. The TEB controller produces linear and angular velocity commands while respecting robot shape, velocity limits, acceleration limits, lookahead time, and obstacle safety margins. During execution, the current odometry and velocity are fed back into the controller at a fixed control rate.

After reaching the goal position, the pipeline applies a final in-place heading adjustment. This makes the final robot pose more deliberate than simply stopping when the translational goal is reached.

Implementation Highlights

The MATLAB implementation keeps the final project workflow in matlab/turtlebot3_navigation.m. The script contains the main configuration, pipeline sequence, and conceptual structure of the project, while reusable operations are implemented under matlab/+mobile_robotics/ so they are easy to inspect, version, and maintain.

Repository Contents

matlab/
MATLAB navigation workflow, source package, occupancy map, and ROS callback helper class.

ros2_ws/
ROS 2 TurtleBot3/Nav2/Gazebo package additions for the indoor simulation world, map, model, and launch files.

docs/
This project report in reStructuredText and PDF form.

Limitations

The project is configured for a known simulated environment and assumes that the TurtleBot3, Gazebo, Nav2, and MATLAB Robotics System Toolbox dependencies are available. The planner and controller parameters are tuned for the included map and may need adjustment for a different robot model, environment scale, or sensor setup.

Conclusion

The project demonstrates a complete mobile-robot navigation stack built around

ROS 2 communication and MATLAB planning tools. It keeps the robotics concepts visible in source form while presenting the work as a maintainable engineering project.